

UNIVERSITÉ DE ROUEN NORMANDIE

UFR des Sciences et Techniques

MASTER INFORMATIQUE - M1 GIL

Année Universitaire 2025-2026

Architecture Distribuée

Système de Gestion de Flotte de Véhicules

Projet Master 1 GIL

Livrable de Conception Architecturale

Auteurs :

Kherbachi Rima

Mouhoubi Leiticia

Encadrants :

Lydia & Luc

20 mars 2026

Table des matières

1	Architecture Microservices	4
1.1	Contexte	4
1.2	Décision	4
1.3	Services identifiés	5
1.4	Conséquences	5
2	Stack Technologique polyglotte	6
2.1	Contexte	6
2.2	Décision	6
2.3	Assignation technologique	6
3	API Gateway GraphQL	6
3.1	Décision	6
3.2	Mécanisme de communication	6
3.3	Conséquences	7
4	Apache Kafka comme bus d'événements	7
4.1	Décision	7
4.2	Mécanisme de communication	7
4.3	Conséquences	8
5	Keycloak pour la gestion d'identité (IAM)	8
5.1	Décision	8
5.2	Preuves d'intégration et de configuration (Import JSON)	8
6	Base de données par service	10
6.1	Décision	10
6.2	Mécanisme et Intégrité des Données	10
7	gRPC pour le service de localisation	10
7.1	Décision	11
7.2	Mécanisme de communication et Flux de données	11
8	OpenTelemetry pour l'observabilité	12
8.1	Décision	12
8.2	Mécanisme de Traçabilité (Tracing)	12
9	Déploiement Kubernetes avec Helm	12
9.1	Contexte	12
9.2	Décision et Mécanisme d'Orchestration	12
10	Saga Pattern pour les transactions distribuées	13
10.1	Contexte	13
10.2	Décision et Mécanisme : La Chorégraphie via Kafka	13

11 Conception et Standardisation des Interfaces (APIs)	14
11.1 Contexte	14
11.2 Décision	14
11.2.1 Documentation et Contrats OpenAPI 3.0	14
11.2.2 Standards de Conception RESTful	14
11.2.3 Sécurité Centralisée via JWT	15
11.2.4 Lien entre APIs Synchrones et Événements Kafka	15
11.3 Conséquences	15
12 Annexe : Modélisation Architecturale (Modèle C4)	16
12.1 Diagramme de Contexte (Niveau 1)	16
12.2 Diagramme de Conteneurs (Niveau 2)	17
12.3 Diagramme de Composants (Niveau 3) — Service Véhicules	18
12.4 Diagramme de Classes (Niveau 4) — Entité Véhicule	19
12.5 Diagramme d'États — Logique Métier du Statut Véhicule	20
13 Modélisation des Données : Approche "Database per Service"	21
13.1 Schéma Global et Découplage	21
14 Sprint 2 : Infrastructure, Conteneurisation et CI/CD	22
14.1 Stratégie de Conteneurisation : Multi-stage Builds	22
14.1.1 Spécificités techniques par microservice	22
14.2 Environnement Local via Docker Compose	23
14.3 Orchestration avec Kubernetes (Minikube)	23
14.4 Intégration et Déploiement Continu (CI/CD)	23
14.5 Validation et Résultats	23

Table des figures

1	Architecture microservices globale : routage synchrone via l'API Gateway, découplage asynchrone par bus d'événements (Apache Kafka) et application du motif "Database per Service".	5
2	Diagramme de séquence illustrant l'agrégation de requêtes par l'API Gateway.	7
3	Diagramme de séquence de la chorégraphie (Pattern Saga) via Kafka, illustrant l'isolation des bases de données.	8
4	Importation des rôles métiers (Realm roles) définissant le RBAC : admin, manager, technicien, utilisateur.	9
5	Clients configurés via JSON, incluant la Gateway et nos 5 microservices métiers.	9
6	Utilisateurs générés avec leurs identifiants de test (admin-fleet, conducteur-fleet, etc.).	10
7	Diagramme de séquence illustrant le pipeline hybride temps réel : ingestion GPS, transfert gRPC (Server Streaming) et restitution via GraphQL Subscriptions.	11
8	Diagramme de Contexte illustrant les interactions avec les acteurs et systèmes externes.	16
9	Diagramme de Conteneurs détaillant l'architecture microservices.	17
10	Diagramme de Composants pour le microservice Véhicules.	18
11	Diagramme de Classes détaillant le modèle de données principal.	19
12	Diagramme d'États illustrant le cycle de vie et les règles métier d'un véhicule.	20
13	Modèle Conceptuel de Données Global : Isolation par microservice et références logiques.	21

1 Architecture Microservices

Numéro	ADR-001
Titre	Architecture Microservices
Statut	PROPOSÉ
Décideurs	Équipe projet complète
Contexte lié	Projet initial

1.1 Contexte

Le système de gestion de flotte doit gérer des domaines fonctionnels distincts (véhicules, conducteurs, maintenance, localisation, événements) avec des exigences différentes en termes de scalabilité, disponibilité et technologie. Il doit être évolutif, maintenable et permettre le travail en parallèle.

1.2 Décision

Nous adoptons une architecture microservices avec 5 services indépendants, chacun disposant de sa propre base de données, communiquant via une API Gateway (GraphQL) et un bus de messages (Kafka).

- **Communication Synchrone** : Le client ne communique **jamais** directement avec les microservices. Toutes les requêtes passent par l'API Gateway (GraphQL) qui dispatche les appels vers les services concernés.
- **Communication Asynchrone** : Les microservices ne s'appellent pas entre eux via HTTP/REST pour les processus métiers. Ils communiquent exclusivement en publiant et consommant des événements via **Apache Kafka**.
- *Exception métier* : L'API Gateway interroge les services en gRPC ou REST de manière synchrone pour agréger les données à la volée avant de les renvoyer au client.

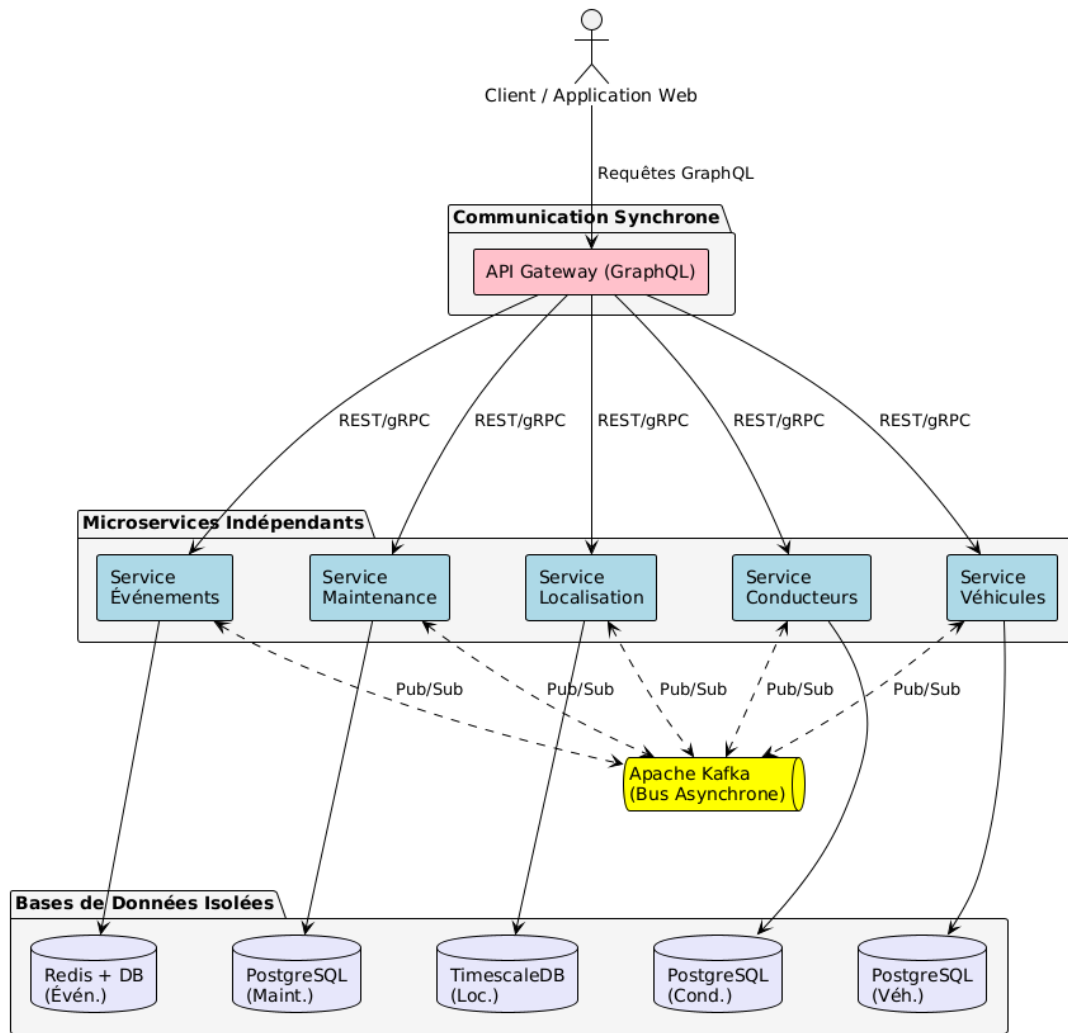


FIGURE 1 – Architecture microservices globale : routage synchrone via l'API Gateway, découplage asynchrone par bus d'événements (Apache Kafka) et application du motif "Database per Service".

1.3 Services identifiés

Service	Responsabilité	Base de données
Véhicules	CRUD parc automobile, disponibilité, historique	PostgreSQL
Conducteurs	Profils, permis, assignations véhicule	PostgreSQL
Maintenance	Planification, suivi interventions, alertes	PostgreSQL
Localisation	Tracking GPS temps réel, géofencing	TimescaleDB
Événements	Alertes, notifications, agrégation événements	Redis + PostgreSQL

1.4 Conséquences

Avantages : Déploiement indépendant, scalabilité ciblée, isolation des pannes, liberté technologique, travail en parallèle.

Inconvénients : Complexité opérationnelle, Saga Pattern nécessaire, latence réseau, surface d'attaque plus large.

2 Stack Technologique polyglotte

Numéro	ADR-002
Titre	Stack Technologique polyglotte
Statut	PROPOSÉ

2.1 Contexte

Chaque microservice a des exigences différentes (robustesse, rapidité I/O, calculs). Une stack unique ne peut pas répondre optimalement à tous les besoins.

2.2 Décision

Nous adoptons une architecture polyglotte avec une infrastructure partagée (Kafka, Docker, Keycloak).

2.3 Assignment technologique

Service	Langage	Justification
Véhicules	Java/Spring Boot	Robustesse, JPA
Conducteurs	Node.js/NestJS	I/O rapide, dev agile
Maintenance	Python/FastAPI	Calculs prévisionnels
Localisation	Node.js/NestJS	Temps réel, I/O intensif
Événements	Python/FastAPI	Agrégation événements

3 API Gateway GraphQL

Numéro	ADR-003
Titre	API Gateway GraphQL
Statut	PROPOSÉ

3.1 Décision

Nous utilisons Apollo Federation (GraphQL) comme API Gateway. Chaque service expose un sous-graphe, Apollo Supergraph les fédère en un schéma unifié. Cela évite au client de gérer de multiples appels et l'agrégation.

3.2 Mécanisme de communication

L'API Gateway est l'unique point d'entrée.

1. L'utilisateur demande le profil d'un conducteur et le statut de son véhicule actuel.
2. La Gateway reçoit la requête GraphQL.
3. Elle interroge simultanément le **Service Conducteurs** et le **Service Véhicules**.
4. Elle assemble la réponse et la renvoie au client.

Cela masque la complexité de l'architecture distribuée au frontend.

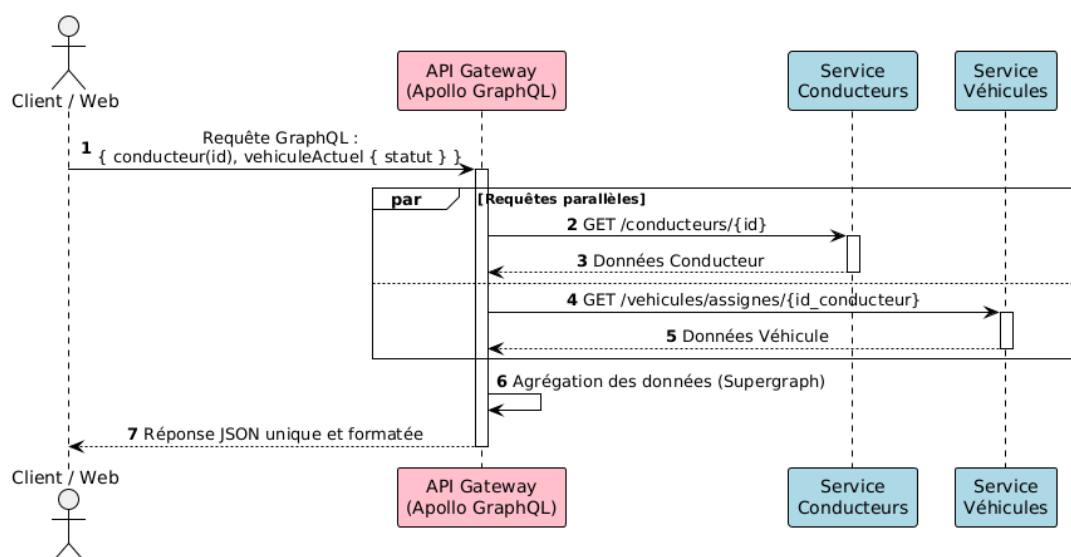


FIGURE 2 – Diagramme de séquence illustrant l'agrégation de requêtes par l'API Gateway.

3.3 Conséquences

Avantages : Réduction des appels réseau, schéma fortement typé, évolution sans breaking changes.

Inconvénients : Complexité de la fédération, gestion du problème N+1 (nécessite DataLoader).

4 Apache Kafka comme bus d'événements

Numéro	ADR-004
Titre	Apache Kafka comme bus d'événements
Statut	PROPOSÉ

4.1 Décision

Nous utilisons Apache Kafka pour toutes les communications asynchrones inter-services (Event-Driven Architecture) afin d'éviter le couplage fort d'une communication REST synchrone.

4.2 Mécanisme de communication

Kafka est la colonne vertébrale des processus métiers inter-services. **Exemple d'interaction type :**

1. Le **Service Véhicules** détecte qu'un véhicule dépasse son kilométrage d'entretien. Il publie un événement `VehiculeNecessiteMaintenance` sur le topic Kafka.
2. Le **Service Maintenance** (qui écoute ce topic) consomme l'événement et crée automatiquement un ticket d'intervention.

3. Le **Service Événements** consomme ce même message pour générer une notification au Manager.

Avantage : Si le Service Maintenance est en panne, le message reste dans Kafka. Il sera traité au redémarrage (résilience).

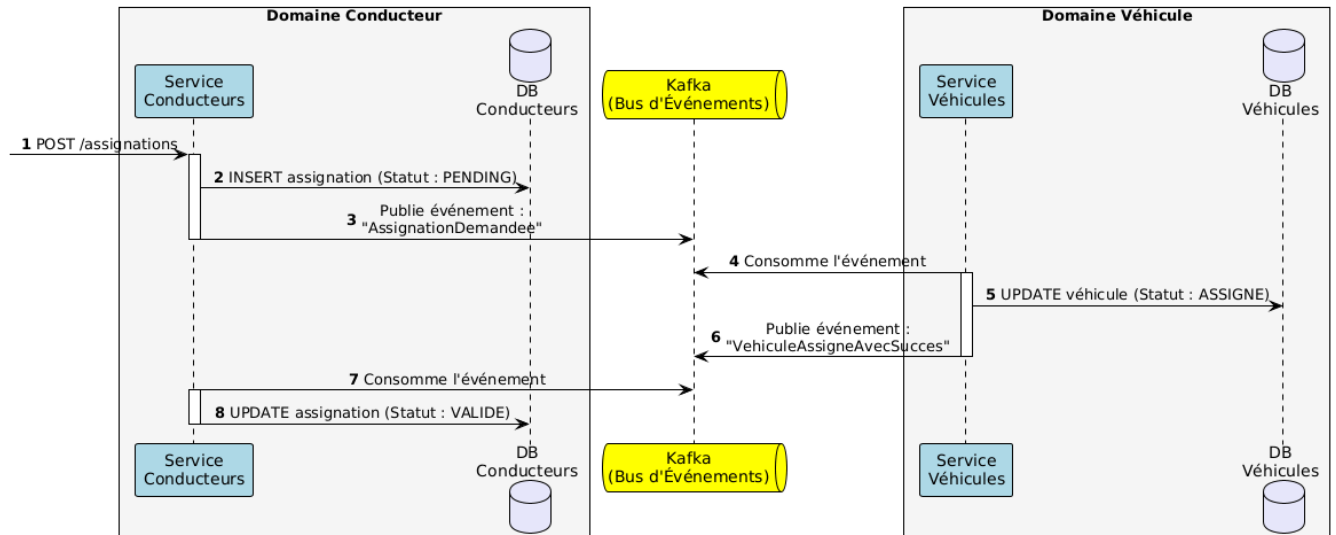


FIGURE 3 – Diagramme de séquence de la chorégraphie (Pattern Saga) via Kafka, illustrant l'isolation des bases de données.

4.3 Conséquences

Avantages : Découplage total, persistance des messages, scalabilité horizontale.

Inconvénients : Éventualité de cohérence, complexité opérationnelle, debugging difficile.

5 Keycloak pour la gestion d'identité (IAM)

Numéro	ADR-005
Titre	Keycloak pour la gestion d'identité (IAM)
Statut	PROPOSÉ

5.1 Décision

Adoption de Keycloak comme solution centralisée. Toutes les authentifications y passent, et Keycloak émet des JWT vérifiés par chaque service, implémentant un modèle RBAC (admin, manager, technicien, utilisateur).

5.2 Preuves d'intégration et de configuration (Import JSON)

Pour garantir la reproductibilité de notre environnement de sécurité, nous avons configuré l'intégralité de notre *Realm* (*fleet-management*) sous forme de fichier JSON (Infrastructure as Code). Ce fichier a été importé avec succès dans notre instance Keycloak.

Les captures ci-dessous prouvent la bonne prise en compte de notre architecture de sécurité :

Realm roles
 Realm roles are the roles that you define for use in the current realm. [Learn more](#)

Search role by name → [Create role](#) [Refresh](#) 1-7 < >

Role name	Composite	Description	
admin	False	Administrateur – accès complet à toute la plateforme et gestion des droits	⋮
default-roles-fleet-management ⓘ	True	role_default-roles	⋮
manager	False	Manager – supervise la flotte, les conducteurs et les alertes	⋮
offline_access	False	role_offline-access	⋮
technicien	False	Technicien – consulte et met à jour le statut des interventions de maintenance	⋮
uma_authorization	False	role_uma_authorization	⋮
utilisateur	False	Conducteur / Utilisateur – consulte ses assignations et utilise les véhicules	⋮

FIGURE 4 – Importation des rôles métiers (Realm roles) définissant le RBAC : admin, manager, technicien, utilisateur.

Chaque composant de notre système est déclaré comme un client indépendant (OpenID Connect), permettant de restreindre et tracer les accès :

Clients
 Clients are applications and services that can request authentication of a user. [Learn more](#)

[Clients list](#) [Initial access token](#) [Client registration](#)

service × → [Create client](#) [Import client](#) [Refresh](#) 1-5 < >

Client ID	Name	Type	Description	Home URL	
driver-service	Service Conducteurs	OpenID Connect	–	–	⋮
event-service	Service Événements	OpenID Connect	–	–	⋮
location-service	Service Localisation	OpenID Connect	–	–	⋮
maintenance-service	Service Maintenance	OpenID Connect	–	–	⋮
vehicle-service	Service Véhicules	OpenID Connect	–	–	⋮

1-5 < >

FIGURE 5 – Clients configurés via JSON, incluant la Gateway et nos 5 microservices métiers.

Enfin, les profils de tests ont été automatiquement provisionnés avec leurs permissions respectives pour la phase de développement :

Users				
Users are the users in the current realm. Learn more				
User list				
Default search		Search user	→	Add user Refresh 1-4
☐ Username	Email	Last name	First name	
☐ admin-fleet	admin@fleet.local	Flotte	Admin	⋮
☐ conducteur-fleet	conducteur@fleet.local	Dupont	Jean	⋮
☐ manager-fleet	manager@fleet.local	Flotte	Manager	⋮
☐ technicien-fleet	tech@fleet.local	Flotte	Technicien	⋮
1-4				

FIGURE 6 – Utilisateurs générés avec leurs identifiants de test (admin-fleet, conducteur-fleet, etc.).

6 Base de données par service

Numéro	ADR-006
Titre	Base de données par service (Database per Service)
Statut	PROPOSÉ

6.1 Décision

Chaque microservice possède sa propre base de données, inaccessible directement par les autres services. Les données partagées transitent via les APIs ou Kafka. PostgreSQL est utilisé pour le relationnel, TimescaleDB pour les séries temporelles, et Redis pour le cache.

6.2 Mécanisme et Intégrité des Données

Puisque les bases de données sont isolées, il est impossible de faire des requêtes SQL JOIN entre les tables de différents domaines (ex : joindre un Conducteur et son Véhicule).

- **Pour la lecture (Requêtes)** : L'API Gateway se charge d'interroger les bases de données via les services respectifs et de faire la "jointure" en mémoire avant de renvoyer le résultat au client.
- **Pour l'écriture (Commandes)** : Lorsqu'une donnée d'un service est modifiée, ce service émet un événement Kafka. Les autres services écoutent cet événement pour mettre à jour leurs propres bases de données (réplication de données ou mise à jour de statuts).

7 gRPC pour le service de localisation

Numéro	ADR-007
Titre	gRPC pour le service de localisation temps réel
Statut	PROPOSÉ

7.1 Décision

Nous utilisons gRPC avec Server Streaming pour la localisation afin de réduire la latence liée au HTTP polling. L'API Gateway traduit les appels gRPC en GraphQL Subscriptions pour le frontend.

7.2 Mécanisme de communication et Flux de données

Nous mettons en place un pipeline hybride gRPC / WebSockets :

1. **Ingestion** : Les boîtiers GPS des véhicules (ou simulateurs) envoient leurs coordonnées à l'API Gateway en continu.
2. **Transfert interne** : L'API Gateway utilise **gRPC (Server Streaming)** pour transférer ces flux de coordonnées au **Service Localisation** avec une latence quasi nulle et une empreinte réseau minimale (Protobuf). Le service stocke ces points dans **TimescaleDB** (optimisé pour les séries temporelles).
3. **Restitution** : L'API Gateway expose une *GraphQL Subscription* (basée sur des WebSockets) au frontend, permettant au navigateur du manager de recevoir les nouvelles positions en direct sans avoir à rafraîchir la page.

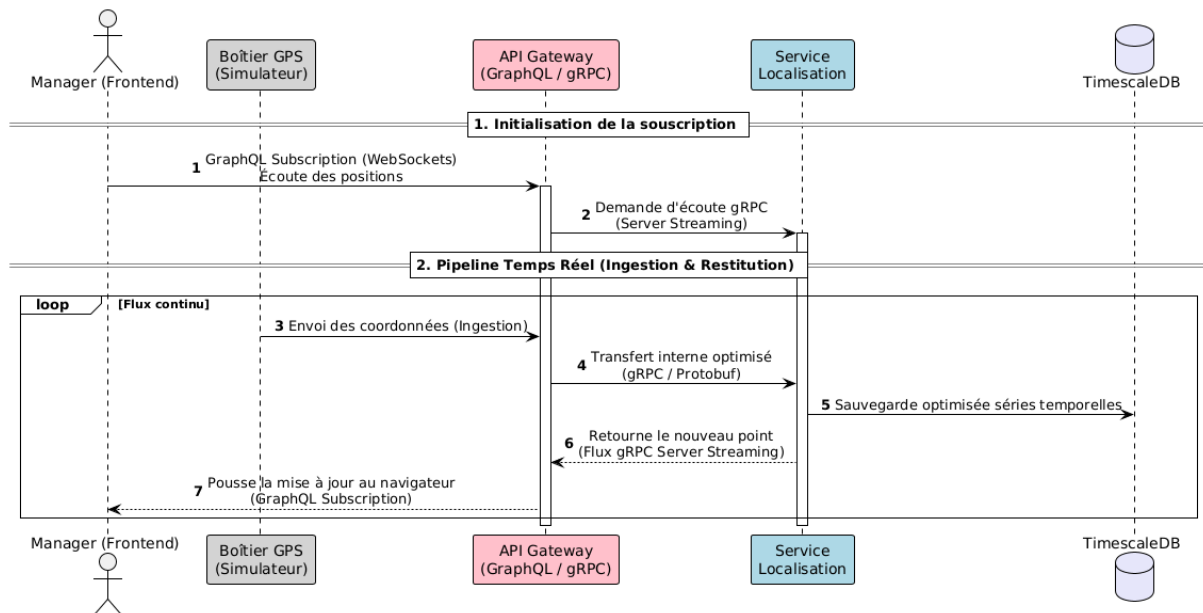


FIGURE 7 – Diagramme de séquence illustrant le pipeline hybride temps réel : ingestion GPS, transfert gRPC (Server Streaming) et restitution via GraphQL Subscriptions.

8 OpenTelemetry pour l'observabilité

Numéro	ADR-008
Titre	OpenTelemetry pour l'observabilité distribuée
Statut	PROPOSÉ

8.1 Décision

Adoption d'OpenTelemetry avec Jaeger (traces), Prometheus/Grafana (métriques) et Loki (logs). Cela permet de corréler les données à travers les 5 microservices. **Métriques clés** : `fleet.vehicles.active`, `fleet.location.updates_per_second`.

8.2 Mécanisme de Traçabilité (Tracing)

Nous utilisons OpenTelemetry pour instrumenter notre code. Le flux fonctionne ainsi :

1. Lorsqu'une requête HTTP arrive sur l'API Gateway, celle-ci génère un identifiant unique de trace (**trace-id**).
2. Cet identifiant est injecté dans les **en-têtes HTTP** lors des appels aux microservices.
3. Si un microservice publie un message Kafka, le **trace-id** est injecté dans les **en-têtes du message Kafka**.
4. Toutes ces traces, associées aux logs (Loki) et aux métriques (Prometheus), sont envoyées à **Jaeger**. Ainsi, nous pouvons visualiser sur un tableau de bord le parcours exact d'une requête, service par service, et identifier quel composant est responsable d'une latence ou d'une erreur.

9 Déploiement Kubernetes avec Helm

Numéro	ADR-009
Titre	Orchestration des conteneurs via Kubernetes et Helm
Statut	PROPOSÉ

9.1 Contexte

Notre architecture compte plus d'une quinzaine de composants (5 microservices, bases de données dédiées, API Gateway, cluster Kafka, Keycloak). Gérer manuellement le cycle de vie, la scalabilité et la résilience de ces composants via de simples scripts ou Docker Compose devient un risque majeur d'instabilité, particulièrement pour un système critique de gestion de flotte.

9.2 Décision et Mécanisme d'Orchestration

Nous adoptons **Kubernetes (K8s)** comme orchestrateur de conteneurs, couplé à **Helm** comme gestionnaire de paquets pour automatiser les déploiements.

Flux réseau et interaction infrastructurelle :

1. **Gestion du trafic externe (Ingress)** : Les requêtes provenant de l'application web frontend sont interceptées par l'*Ingress Controller* de Kubernetes. Celui-ci fait office de point d'entrée unique et de routeur intelligent pour rediriger le flux vers le *Service* Kubernetes de l'API Gateway.
2. **Découverte de services (Service Discovery)** : En interne, les microservices ne communiquent pas via des adresses IP statiques (qui changent à chaque redémarrage). Kubernetes maintient un DNS interne (CoreDNS). Par exemple, pour que l'API Gateway contacte le service de maintenance, elle appelle simplement le nom d'hôte `maintenance-service.default.svc.cluster.local`.
3. **Haute disponibilité (Auto-healing)** : Des sondes de vie (*Liveness* et *Readiness Probes*) surveillent chaque microservice. Si le **Service Véhicules** (développé en Spring Boot) devient injoignable ou rencontre une erreur critique, Kubernetes redémarre automatiquement son *Pod* sans intervention humaine pour garantir la continuité de service.
4. **Déploiement reproductible avec Helm** : Plutôt que de gérer des dizaines de fichiers YAML complexes, nous utilisons des *Charts* Helm. Cela nous permet de packager notre application : une seule commande déploie l'intégralité de l'infrastructure de manière identique entre nos environnements de développement, de test et de production.

10 Saga Pattern pour les transactions distribuées

Numéro	ADR-010
Titre	Saga Pattern (Chorégraphie) pour la cohérence des données
Statut	PROPOSÉ

10.1 Contexte

Dans une architecture distribuée, une transaction métier implique souvent plusieurs microservices et leurs bases de données respectives. Il est nécessaire de garantir la cohérence des données (ex : assignation d'un véhicule) sans recourir à un verrouillage centralisé, sous peine de créer un point de défaillance unique et de réduire les performances.

10.2 Décision et Mécanisme : La Chorégraphie via Kafka

Nous implémentons le pattern Saga selon le modèle de la **chorégraphie** (sans orchestrateur central). Les microservices réagissent de manière autonome aux événements publiés dans Kafka.

Scénario d'assignation d'un véhicule :

1. Le **Service Conducteurs** enregistre l'assignation (état *PENDING*) et publie l'événement `AssignationDemandee`.
2. Le **Service Véhicules** consomme l'événement, vérifie si le véhicule est *DISPONIBLE*, le passe en *ASSIGNE*, et publie `VehiculeAssigneAvecSucces`.
3. Le **Service Conducteurs** consomme ce succès et passe l'assignation en état *VALIDE*.

4. **Gestion d'erreur (Rollback)** : Si le véhicule était en maintenance (ou indisponible), le Service Véhicules publie `EchecAssignmentVehicule`. Le Service Conducteurs consomme cette erreur et exécute une **transaction compensatoire** (passage de l'assignation en *REJETEE* ou *ANNULEE*).

11 Conception et Standardisation des Interfaces (APIs)

Numéro	ADR-011
Titre	Conception et Standardisation des Interfaces (APIs)
Statut	PROPOSÉ

11.1 Contexte

Dans le cadre de notre architecture distribuée, la définition claire des contrats d'interface est primordiale. Sans règles communes, chaque microservice risquerait d'implémenter sa propre logique de communication, rendant l'intégration avec l'API Gateway et le frontend complexe et fragile.

11.2 Décision

Nous avons établi un ensemble de principes directeurs stricts pour nos APIs REST et notre Gateway GraphQL, adoptant une approche "API-First" pour garantir l'indépendance des équipes et la robustesse des communications.

11.2.1 Documentation et Contrats OpenAPI 3.0

Chaque microservice métier (Véhicules, Conducteurs, Maintenance, Événements) expose ses propres endpoints REST selon le standard OpenAPI 3.0.3. Cette approche nous permet :

- De générer automatiquement une documentation interactive (Swagger UI).
- De définir rigoureusement les schémas de données attendus en entrée (ex : `CreateVehiculeDto`) et en sortie, garantissant la validation des données avant tout traitement métier.

11.2.2 Standards de Conception RESTful

Afin de garantir une uniformité sur l'ensemble du système, nos APIs respectent des standards stricts :

- **Pagination et Filtrage** : Tous les endpoints retournant des collections (ex : `GET /vehicules`) implémentent nativement une pagination via les paramètres de requête `page` et `limit`, et permettent le filtrage par `statut` ou `actif`.
- **Verbes HTTP sémantiques** : Utilisation stricte de `POST` pour la création, `PUT` pour la mise à jour complète, et `PATCH` pour les actions métiers spécifiques (ex : `PATCH /interventions/{id}/terminer`).
- **Soft Deletion** : Les requêtes `DELETE` n'entraînent pas de suppression physique en base de données afin de préserver l'intégrité de l'historique et des statistiques, mais appliquent une désactivation logique.

11.2.3 Sécurité Centralisée via JWT

La couche API agit comme la première ligne de défense de notre système. Tous les points d'entrée (à l'exception des sondes de vie pour Kubernetes) sont protégés par le schéma de sécurité `oauth2Keycloak`. Les microservices valident la signature du Token JWT et s'assurent que l'appelant possède les *scopes* adéquats (admin, manager, technicien, utilisateur) avant d'autoriser l'accès aux ressources.

11.2.4 Lien entre APIs Synchrones et Événements Kafka

Il est important de noter que nos APIs ne se contentent pas d'écrire en base de données. Plusieurs endpoints critiques agissent comme des déclencheurs pour notre architecture événementielle (Saga Pattern). Par exemple :

- L'appel à `POST /assignments` génère une réponse synchrone (201 Created) pour le client, puis publie immédiatement un événement asynchrone `fleet.conducteurs.assignment` dans Kafka.
- L'appel à `PATCH /vehicules/{id}/statut` met à jour la base de données PostgreSQL du service Véhicules et émet un événement `fleet.vehicules.statut` pour informer le service de Maintenance.

11.3 Conséquences

Avantages : Contrats clairs facilitant le travail en parallèle, validation robuste des données en amont, sécurité unifiée, et documentation toujours à jour.

Inconvénients : Effort initial de conception plus important, rigueur exigée pour maintenir les schémas OpenAPI synchronisés avec le code.

12 Annexe : Modélisation Architecturale (Modèle C4)

Afin d'illustrer les choix d'architecture documentés dans les ADR précédents, nous nous appuyons sur le modèle C4.

12.1 Diagramme de Contexte (Niveau 1)

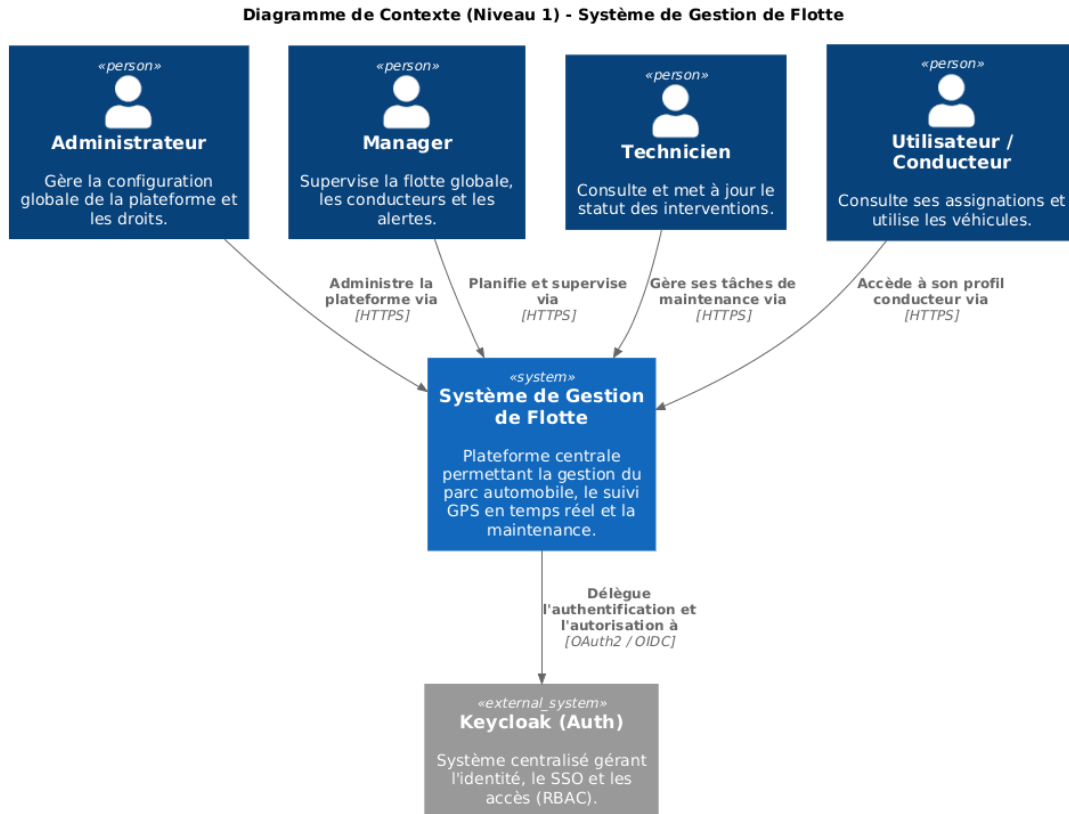


FIGURE 8 – Diagramme de Contexte illustrant les interactions avec les acteurs et systèmes externes.

Explication : Ce premier niveau illustre les interactions globales.

- **Les acteurs :** Les quatre rôles du système interagissent avec la plateforme centrale selon des droits spécifiques (RBAC).
- **Le système central :** Il représente l'application dans son ensemble.
- **Le système externe :** Keycloak gère l'identité (IAM) via OAuth2 / OIDC.

12.2 Diagramme de Conteneurs (Niveau 2)

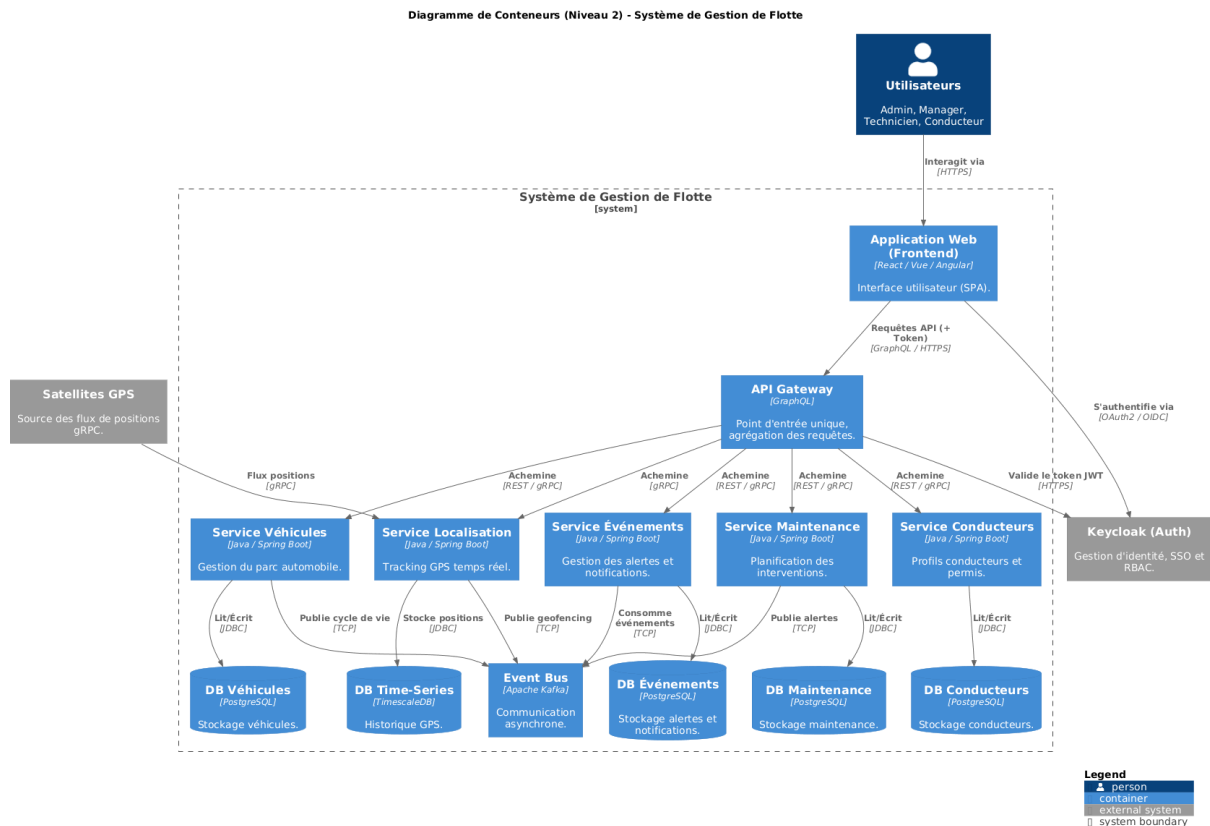


FIGURE 9 – Diagramme de Conteneurs détaillant l'architecture microservices.

Explication : Ce niveau révèle l'architecture microservices.

- **Le point d'entrée :** L'Application Web communique avec l'API Gateway (GraphQL).
- **Les Microservices :** Les 5 services métiers indépendants sont clairement séparés.
- **Isolation des données :** Illustre l'utilisation de PostgreSQL et TimescaleDB.
- **Communication Asynchrone :** L'Event Bus (Apache Kafka) montre les flux de publication/consommation.

12.3 Diagramme de Composants (Niveau 3) — Service Véhicules

Diagramme de Composants (Niveau 3) - Service Véhicules (Java/Spring Boot)

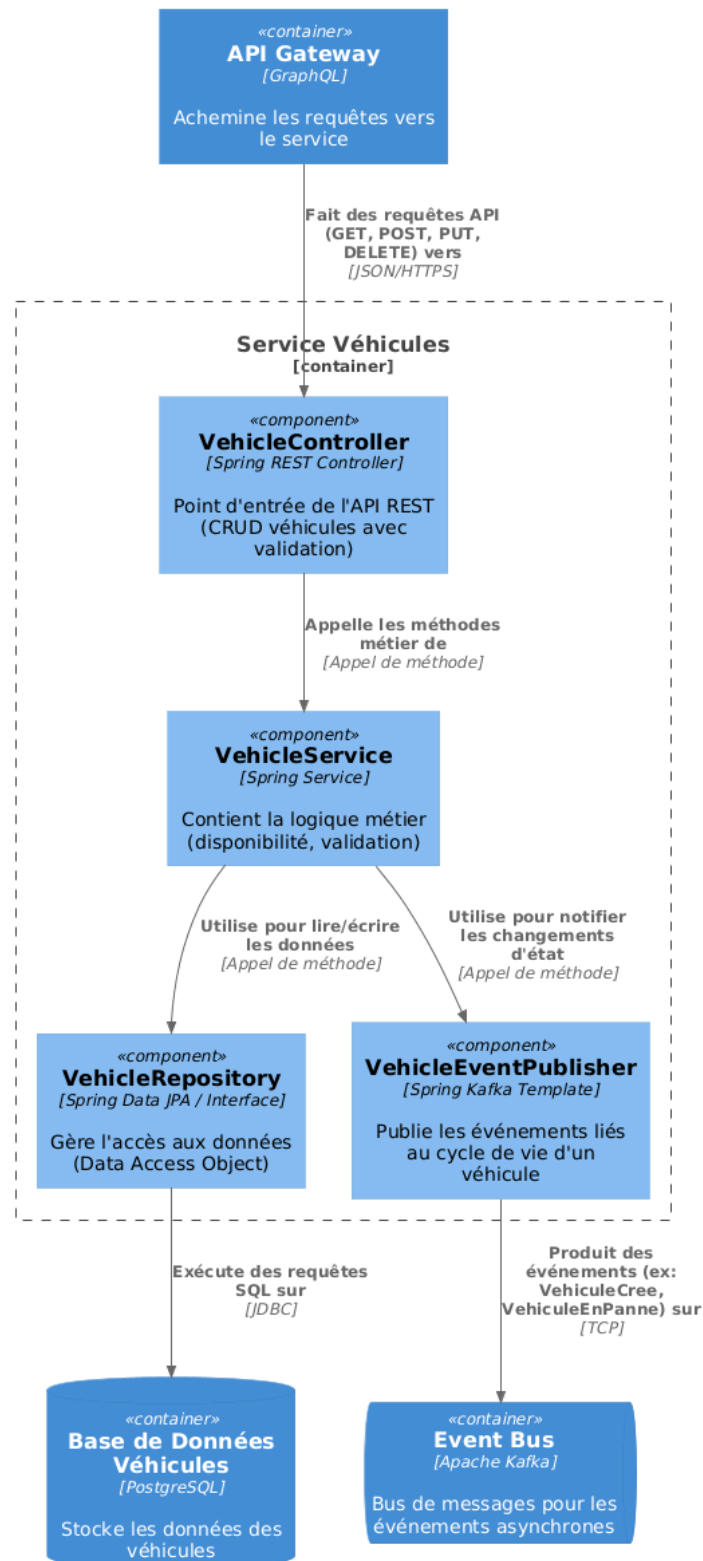


FIGURE 10 – Diagramme de Composants pour le microservice Véhicules.

Explication : Ce diagramme zoome à l'intérieur du **Service Véhicules** (Java/Spring Boot). Il illustre la séparation :

- **VehicleController** : Gère les requêtes HTTP.
- **VehicleService** : Contient la logique métier.
- **VehicleRepository** : Gère l'accès isolé à la base PostgreSQL.
- **VehicleEventPublisher** : Gère l'envoi des messages vers Kafka.

12.4 Diagramme de Classes (Niveau 4) — Entité Véhicule

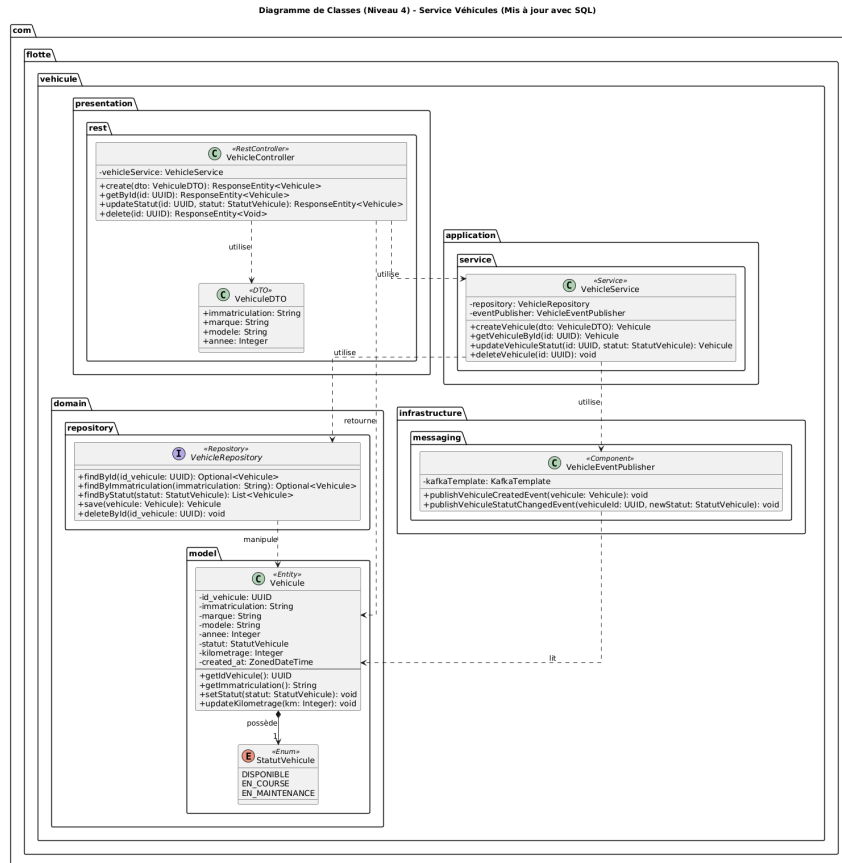


FIGURE 11 – Diagramme de Classes détaillant le modèle de données principal.

Explication : Représente la structure conceptuelle des données (immatriculation, marque, modèle) et met en avant l'énumération **StatutVehicle**. Le changement de ce statut est le déclencheur principal des événements Kafka.

12.5 Diagramme d'États — Logique Métier du Statut Véhicule

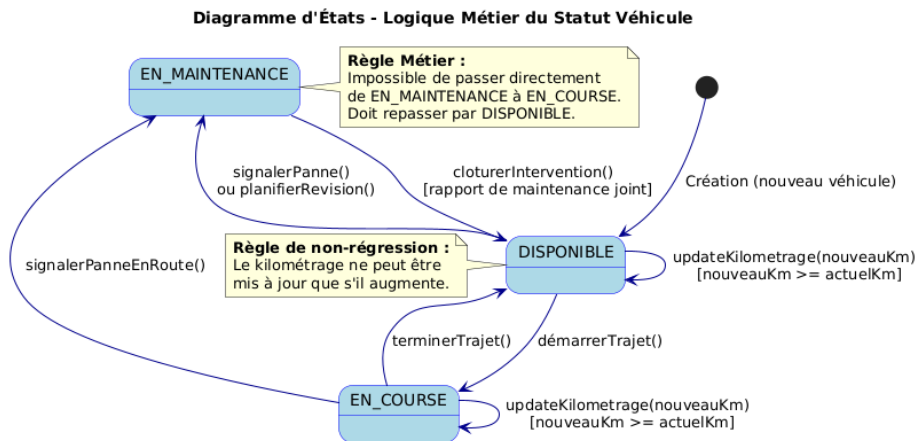


FIGURE 12 – Diagramme d'États illustrant le cycle de vie et les règles métier d'un véhicule.

Explication : Ce diagramme modélise la logique métier (Machine à états) régissant le cycle de vie du statut d'un véhicule au sein du service.

- **Transitions strictes :** Un véhicule ne peut passer EN_COURSE que s'il est préalablement DISPONIBLE. Il n'y a pas de passage direct de la maintenance à la course.
- **Verrouillage métier :** Un véhicule EN_MAINTENANCE doit obligatoirement faire l'objet d'une clôture d'intervention (avec rapport joint) pour redevenir DISPONIBLE.
- **Règle de non-régression :** La boucle sur les états actifs illustre la mise à jour du kilométrage, qui est soumise à une règle stricte : la nouvelle valeur doit toujours être supérieure ou égale à la valeur actuelle.

13 Modélisation des Données : Approche "Database per Service"

Conformément à notre décision d'architecture (ADR-006), notre système s'appuie sur le pattern *Database per Service*. Cette approche garantit un couplage faible et une autonomie maximale pour chaque domaine métier.

13.1 Schéma Global et Découplage

Le diagramme ci-dessous illustre la modélisation globale du système. Bien que les données soient réparties sur 5 bases de données distinctes (PostgreSQL et TimescaleDB), elles maintiennent une cohérence logique.

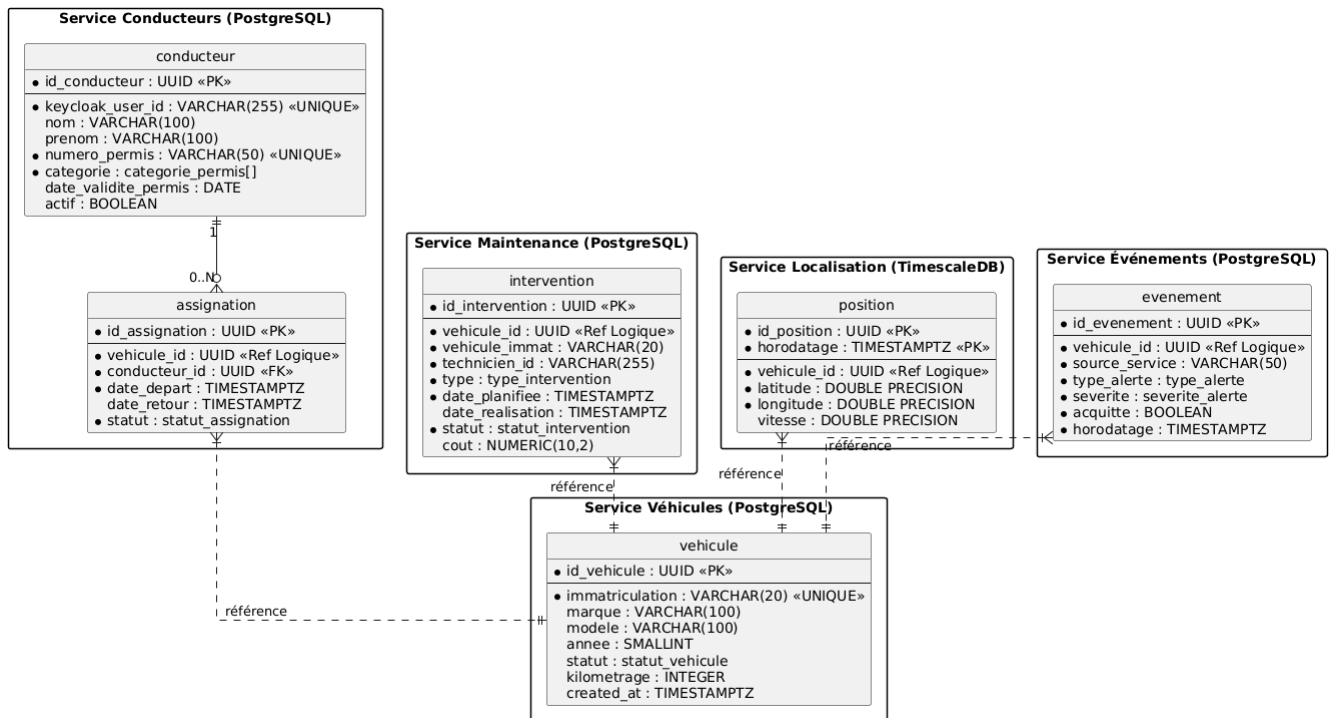


FIGURE 13 – Modèle Conceptuel de Données Global : Isolation par microservice et références logiques.

Principes appliqués dans cette modélisation :

- **Absence de clés étrangères inter-services** : Les bases de données ne partagent aucune contrainte d'intégrité référentielle (Foreign Key) stricte. Les relations inter-domaines s'effectuent via des **références logiques** (Soft Links), généralement matérialisées par des identifiants uniques universels (UUID).
- **Respect des Bounded Contexts** : Chaque service est la source de vérité unique pour son domaine. Par exemple, l'entité **Assignation** appartient exclusivement au **Service Conducteurs**. Le **Service Véhicules** n'a pas connaissance de l'identité du conducteur ; il se contente de mettre à jour son propre état (**statut** = 'en_cours') en réagissant aux événements asynchrones (Kafka) émis par le Service Conducteurs.
- **Optimisation des lectures** : Certaines données critiques pour l'affichage sont dénormalisées, comme **vehicule_immat** dans le **Service Maintenance**, afin d'éviter

des appels réseau synchrones coûteux lors de la consultation des historiques d'intervention.

- **Séries Temporelles** : Le **Service Localisation** exploite une *Hypertable* partitionnée via TimescaleDB, avec une clé primaire composite incluant l'horodatage, répondant ainsi aux exigences de haute performance pour l'ingestion de coordonnées GPS.

14 Sprint 2 : Infrastructure, Conteneurisation et CI/CD

Ce chapitre présente le travail réalisé durant la deuxième semaine du projet, axé sur la mise en place de l'infrastructure complète de développement et de déploiement. L'objectif était de concevoir une architecture robuste et d'automatiser le cycle de vie logiciel.

14.1 Stratégie de Conteneurisation : Multi-stage Builds

Pour chaque microservice, nous avons mis en place des stratégies de conteneurisation optimisées en utilisant des Dockerfiles multi-stage. Cette approche consiste à séparer le processus en deux étapes distinctes dans un même fichier :

- **Étape 1 (Build)** : Une image complète avec tous les outils de compilation (Maven pour Java, npm pour Node.js, pip pour Python) est utilisée pour compiler le code source et produire les artefacts.
- **Étape 2 (Production)** : Une image beaucoup plus légère, sans les outils de build, récupère uniquement les artefacts compilés. Cela réduit considérablement la taille finale de l'image.

D'un point de vue sécurité, tous nos Dockerfiles créent un utilisateur non-root dédié (spring, nodejs, appuser) et basculent sur cet utilisateur avant de lancer l'application. Cela suit le principe du moindre privilège pour limiter les risques en cas de compromission.

14.1.1 Spécificités techniques par microservice

- **vehicule-service (Java/Spring Boot)** : L'étape de build copie d'abord le `pom.xml` pour télécharger les dépendances en cache, évitant de tout retélécharger à chaque changement. L'image de production utilise `eclipse-temurin:17-jre-alpine` (le JRE au lieu du JDK complet), divisant la taille de l'image par 3.
- **conducteur-service (Node.js/NestJS)** : L'image de production installe uniquement les dépendances de production via `npm ci --omit=dev`, excluant les outils de développement.
- **Services Python (FastAPI)** : L'étape de build installe les dépendances via `pip install --user`. L'image de production copie ces dépendances déjà installées. La variable `PYTHONUNBUFFERED=1` est définie pour afficher les logs sans mise en tampon.
- **localisation-service (Node.js/gRPC)** : Les fichiers `proto` définissant le contrat gRPC ne sont pas compilés par TypeScript. Ils sont donc copiés manuellement dans l'image de production pour que le serveur puisse les lire au démarrage.

14.2 Environnement Local via Docker Compose

Un fichier `docker-compose.yml` a été créé pour lancer l'ensemble de l'infrastructure localement sans Kubernetes, facilitant les tests rapides. Il orchestre :

- **L'infrastructure** : PostgreSQL, TimescaleDB, Redis, Kafka et Keycloak partageant le même réseau interne `fleet-network`.
- **Les microservices** : Les 5 services métiers sont configurés avec `depends_on` pour garantir que les bases de données et Kafka démarrent avant eux.

14.3 Orchestration avec Kubernetes (Minikube)

Les manifestes Kubernetes ont été organisés dans un dossier `k8s/` pour définir l'état désiré de l'infrastructure sur un cluster Minikube.

- **Namespace et Sécurité** : Toutes les ressources sont isolées dans le namespace `fleet-management`. Les informations sensibles (mots de passe, URLs) sont stockées dans des Secrets encodés en base64 et référencés dans les Deployments.
- **Haute Disponibilité** : Les Deployments configurent 2 replicas par service, utilisent la politique `imagePullPolicy: Never` pour les images locales, et intègrent des sondes de santé (*Liveness Probe*) ainsi que des limites de ressources CPU/mémoire.
- **Routage (Ingress)** : Les requêtes extérieures passent par un Ingress unique configuré avec le hostname `api.fleet.local`, qui redirige le trafic vers les services internes de type ClusterIP via Nginx.
- **Kafka via Strimzi** : L'opérateur open-source Strimzi a été privilégié pour déployer Kafka via une Custom Resource Definition (CRD), offrant une meilleure intégration avec Kubernetes.
- **Observabilité** : Une stack complète (Prometheus, Grafana, Loki, Jaeger, Open-Telemetry Collector) a été déployée pour tracer les requêtes, agréger les logs et surveiller les métriques.

14.4 Intégration et Déploiement Continu (CI/CD)

Le processus a été fiabilisé en automatisant le cycle de vie logiciel via un pipeline CI/CD GitHub Actions. Ce pipeline gère les étapes de construction (Build), de validation et de tests de l'application.

14.5 Validation et Résultats

Le déploiement intégral a été validé grâce à des outils de diagnostic comme FreeLens. L'état du cluster Kubernetes post-déploiement montre la bonne santé de l'infrastructure avec 30 pods actifs en état *Running*, 10 Deployments disponibles, et 10 Stateful Sets pour la gestion des composants persistants (bases de données). L'architecture polyglotte et distribuée est ainsi pleinement fonctionnelle sur son environnement orchestré.